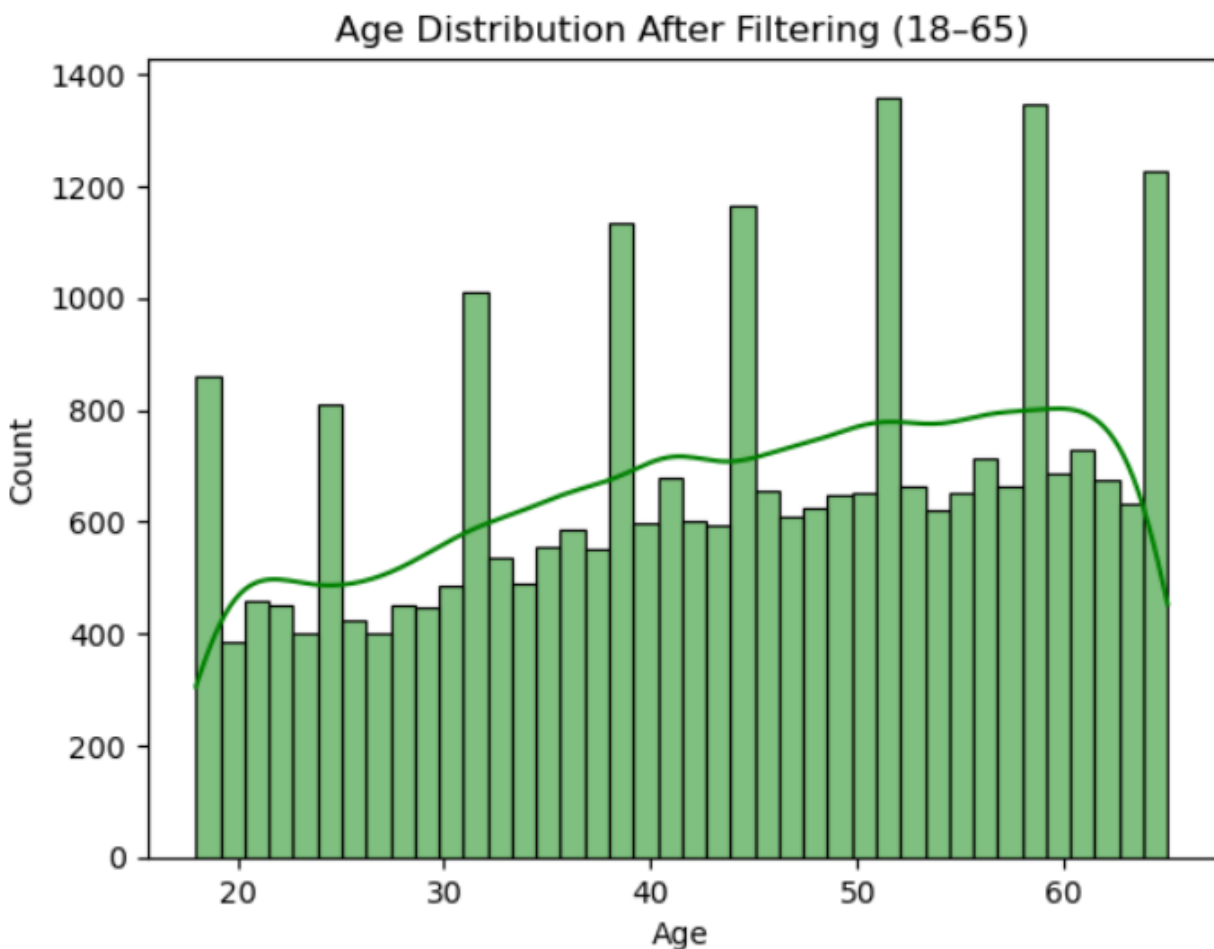


1. Loading and Filtering the Dataset

The first step was importing the dataset using `pandas.read_csv()` into a DataFrame. The original dataset contained a very high number of variables (618) and tens of thousands of observations.

```
df = pd.read_csv("ESS10.csv", sep=',', low_memory=False)
```

Immediately after loading, the dataset was filtered to include only working-age adults between 18 and 65 years.



The dataset was reduced to 27,224 observations. This step was crucial because individuals outside this age range are typically not actively participating in the labor force, which could introduce noise or bias into employment classification models.

At this point, the dataset contains a combination of survey responses and structural metadata that enables both macro-level comparability and micro-level econometric modeling. The columns include:

- Dataset Identifiers and Metadata: name, essround, edition, proddate indicate the exact wave and version of the dataset. These identifiers are important for reproducibility and long-term citation of the dataset version.
- Respondent-Level Identification: idno uniquely identifies individuals (anonymized), while cntry provides their country code.
- Sampling Weights and Adjustment Factors: dweight, pspwght, pweight, anweight are used for cross-national comparisons, multi-level regression models, and proper variance estimation in complex survey designs.
- Interview Timing and Process Markers: Timestamps such as vinwe, inwde, jinws, jinwe provide information about when the interview occurred and how long it lasted. These are useful for checking fatigue effects or interviewing mode bias. The inwtm column, for instance, gives the total interview time in minutes.
- Survey Operation and Sample Design Variables: mode (interview method), domain, prob, stratum, and psu are key for reproducing stratification or clustering structures in advanced statistical models.

This section of the data is not only foundational for filtering and validation, but also enables weighting, replication of design-based inference, and correction for survey mode effects. By retaining these columns through preprocessing, we preserved the flexibility to run either pooled cross-sectional or country-specific models, with or without weight adjustments.

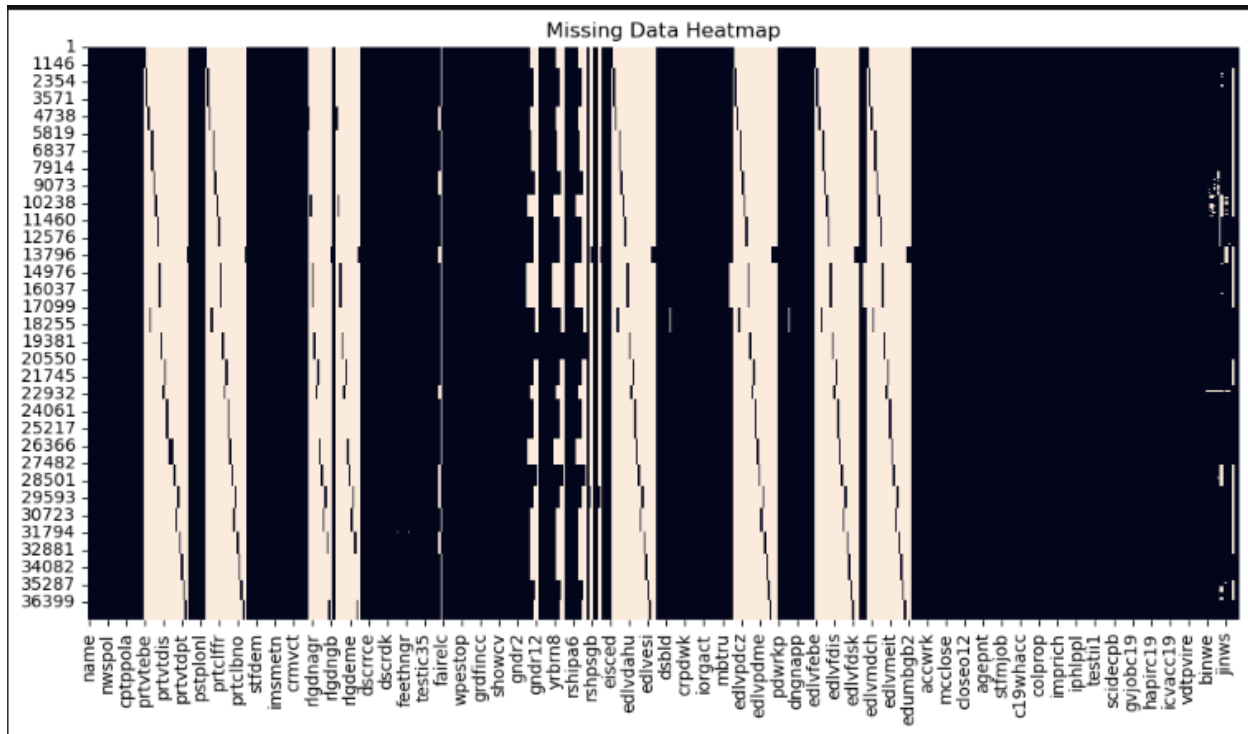
We also inspected the dataset visually and programmatically (df.head(), df.describe()) to confirm data quality, completeness of metadata, and presence of expected value ranges before any deeper preprocessing commenced. This meticulous attention to initial structure, sampling details, and filtering ensures a robust base for every subsequent econometric model that follows.

df.head()

	name	essround	edition	proddate	idno	cntry	dweight	pspwght	pweight	anweight	...	vinwe	inwde	jinws	jinwe	inwtm	mode	domain
1	ESS10e03_2	10	3.2	02.11.2023	10053	BE	1.047643	0.888635	0.718075	0.638107	...	2022-04-08 11:07:00	2022-04-08 11:10:00	2022-04-08 11:07:00	2022-04-08 11:10:00	54.0	2	2.0 0.
2	ESS10e03_2	10	3.2	02.11.2023	10055	BE	1.087741	0.722811	0.718075	0.519033	...	2022-05-20 11:08:00	2022-05-20 11:10:00	2022-05-20 11:08:00	2022-05-20 11:10:00	77.0	1	2.0 0.
3	ESS10e03_2	10	3.2	02.11.2023	10062	BE	0.909910	1.005565	0.718075	0.722072	...	2022-05-22 13:58:00	2022-05-22 13:59:00	2022-05-22 13:58:00	2022-05-22 13:59:00	55.0	1	1.0 0.
4	ESS10e03_2	10	3.2	02.11.2023	10064	BE	0.918949	0.638705	0.718075	0.458639	...	2022-05-18 11:44:00	2022-05-18 11:45:00	2022-05-18 11:44:00	2022-05-18 11:45:00	55.0	1	1.0 0.
5	ESS10e03_2	10	3.2	02.11.2023	10077	BE	0.913740	0.706166	0.718075	0.507080	...	2022-03-17 18:24:00	2022-03-17 18:25:00	2022-03-17 18:24:00	2022-03-17 18:25:00	68.0	1	1.0 0.

5 rows × 618 columns

Among the 618 columns in our dataset, a significant number exhibit a high prevalence of missing data, and many of these variables appear to be extraneous to the core objectives of our analysis. The accompanying graph clearly illustrates the widespread distribution of missing values, thereby underscoring the need for a rigorous variable selection process and potential imputation strategies.



After filtering to the working-age population and analyzing the codebook, we reduced the full dataset from 618 columns to a focused subset of 23 variables directly relevant to our goal: modeling employment status (pdwrk). These variables were selected based on their conceptual relevance, econometric significance, and data quality as indicated in the ESS10 codebook. The selection included demographics, household context, education, income perception, and values or behaviors that may influence employment status.

```
df_copy = df.copy()

columns = ["pdwrk", "agea", "gndr", "agepnt", "ttminpnt", "acchome", "chldo12", "hhlio12", "marsts", "domicil", "hinctnta",
           "health", "hlthhmp", "ipadvnt", "hincfel", "sclact", "ipcrtiv", "ipmodst", "hincsrca", "mnactp"]

df_train = df[columns].copy()
```

From the 618 original columns, we selected 20 key variables that provide the theoretical and empirical foundation for modeling employment status. These variables were chosen based on:

- Relevance to labor market participation, Coverage of core domains: demographics, health, income, family structure, values, and social engagement, completeness, interpretability, and standardization

Each selected variable contributed to explaining employment likelihood as follows:

- Target Variable: pdwrk — Binary variable indicating if the respondent is currently working for pay.
- Demographics: agea, gndr, marsts, domicil — Key baseline controls affecting labor supply decisions.
- Family Responsibilities: agepnt, chldo12, hhllo12 — Capture caregiving duties and parental status.
- Time Use: ttminpnt — Time on personal care, reflecting constraints or unmeasured disability.
- Socioeconomic Status: acchome, hinctnta, hincfel, hincsrca, mnactp — Indicators of financial conditions and income sources.
- Health: health, hlthhmp — Self-rated health and functional limitations, strong predictors of employability.
- Values and Psychology: ipadvnt, imprich, ipcrtiv, ipmodst, — Reflect intrinsic motivation, life outlook, and work-related values.
- Social Engagement: sclact — Gauge social capital and integration, which may affect employment opportunities.

Why not the rest? The other 595 variables were excluded due to: country-specific or irrelevant content, rare modules with high missingness, redundancy with selected indicators, interview logistics unrelated to employment behavior.

After narrowing down our dataset to the 20 most pertinent variables, we reexamined the incidence of missing values by computing their percentage relative to the total dataset. This refined analysis provided a clearer perspective on the quality and completeness of our data, facilitating targeted strategies for addressing gaps and ensuring the robustness of subsequent statistical analyses. In addition, breaking down missing data by percentage enables us to prioritize variables for imputation or exclusion, ultimately enhancing analytical precision and contributing to more reliable model outcomes.

3. Identifying and Removing Near-Zero Variance Features

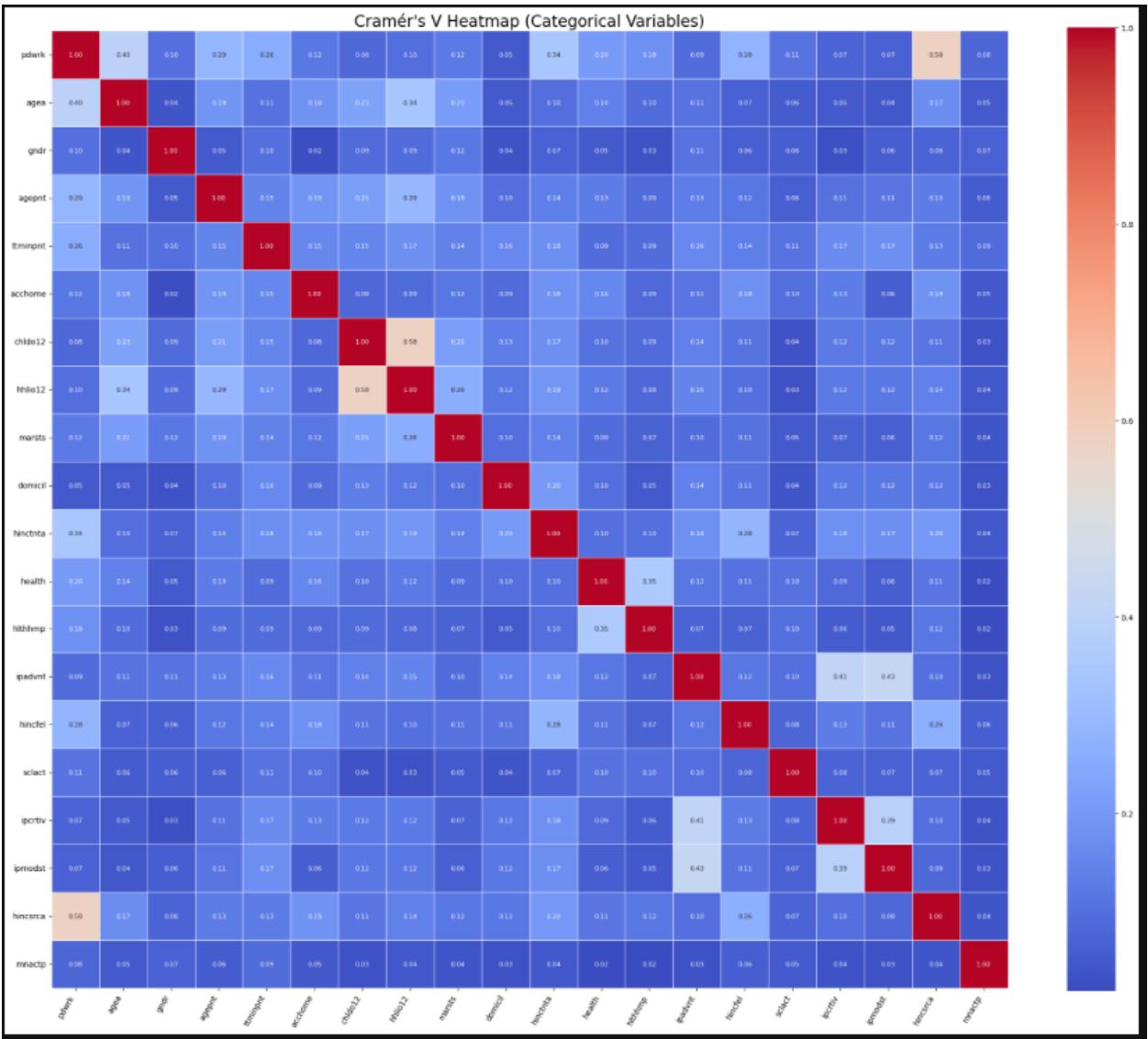
To reduce redundancy and avoid predictors that would contribute minimal explanatory power to the model, we applied a check for near-zero variance (NZV) features. Variables with extremely low variance can cause instability in estimation or act as noise. Using `sklearn.feature_selection.VarianceThreshold`, we assessed the variance of each categorical feature (after preliminary conversion where needed). In this case, no variables were flagged as near-zero due to prior thoughtful selection from ESS10; the 23 selected features already demonstrated sufficient spread based on the codebook. All 23 variables passed the variance threshold. This confirmed the preselected set was already information-rich and suitable for further analysis.

4. Assessing Multicollinearity Among Categorical Features: Cramér's V

We used Cramér's V matrix to examine multicollinearity across categorical variables. This is essential for discrete data where Pearson correlation is not appropriate. The heatmap visualization helped highlight pairs of highly correlated variables, suggesting redundancy or potential collinearity concerns.

	sclact	ipertiv	ipmodst	hincsrca	mnactp
pdwrk	0.108	0.073	0.067	0.579	0.078
agea	0.056	0.052	0.045	0.172	0.053
gnr	0.061	0.029	0.059	0.076	0.065
agepnt	0.063	0.106	0.111	0.127	0.056
ttminpnt	0.110	0.165	0.174	0.127	0.089
acchome	0.097	0.129	0.060	0.186	0.049
chldo12	0.038	0.124	0.120	0.114	0.033
hhllo12	0.033	0.123	0.119	0.138	0.042
marsts	0.051	0.069	0.064	0.116	0.041
domicil	0.041	0.119	0.122	0.119	0.034
hinctnta	0.074	0.178	0.175	0.199	0.045
health	0.104	0.087	0.059	0.111	0.019
hlthhmp	0.102	0.057	0.047	0.118	0.022
ipadvnt	0.099	0.412	0.428	0.101	0.033
hincfel	0.078	0.128	0.108	0.262	0.059
sclact	1.000	0.076	0.066	0.075	0.052
ipertiv	0.076	1.000	0.387	0.097	0.035
ipmodst	0.066	0.387	1.000	0.084	0.031
hincsrca	0.075	0.097	0.084	1.000	0.041
mnactp	0.052	0.035	0.031	0.041	1.000

The matrix showed modest correlations among a few value-based variables (e.g., ipadvnt and ipcrtiv) but none exceeded thresholds warranting exclusion. This confirmed our variable independence assumptions were broadly met. Here graph version of the matrix but it is not so clear so that's above you have accurate numeric values of this heatmap.



5. Cleaning Coded Missing Values

ESS10 uses a set of predefined special codes in many of its categorical variables to indicate non-substantive responses.

These include:

7, 77 — Refusal to answer

8, 88 — Don't know / Cannot say

9, 99, 999 — No answer / Not applicable / Missing

These codes are not missing in the technical sense from a programming perspective, but represent valid placeholders for non-responses in the survey methodology. However, if not addressed, they may severely distort summary statistics and bias model coefficients by being treated as actual values. We performed a systematic replacement of these codes using `pandas.replace()` to convert them into true missing values (NaN) that are appropriately handled in data analysis pipelines.

	agepnt	ttminpnt	chldo12
count	14749.000000	14749.000000	14749.000000
mean	68.982982	107.108957	0.830904
std	11.027823	265.861090	1.098124
min	32.000000	0.000000	0.000000
25%	61.000000	10.000000	0.000000
50%	69.000000	25.000000	0.000000
75%	77.000000	80.000000	2.000000
max	90.000000	5760.000000	6.000000

This operation was carefully verified using frequency tables and null-value counts to ensure no unintentional values were converted. The rationale behind this step is threefold. Statistical Integrity – Ensures that all missing values are treated consistently across variables and excluded from calculations where appropriate. Modeling Readiness – Most machine learning and statistical models in Python (e.g., `scikit-learn`, `statsmodels`) require missing data to be explicitly marked as NaN. Transparency – This approach aligns with recommended ESS10 data handling practices and improves reproducibility of research findings. This step allowed us to cleanly identify incomplete observations and set up the dataset for proper downstream encoding, correlation analysis, and eventual row-wise deletion of incomplete entries. This improved the integrity of summary statistics, correlation analysis, and model input readiness.

6. Encoding Categorical Variables

To prepare our dataset for binary classification models such as Logit and Probit (to be implemented in R), we needed to convert categorical variables into a numerical format suitable for statistical computation. Encoding is critical for algorithms to correctly interpret qualitative input data. However, a one-size-fits-all approach is not appropriate here. We adopted a hybrid encoding approach — using both Label Encoding and One-Hot Encoding depending on the nature and distribution of each variable. The logic was as follows: Binary Label Encoding for Dichotomous Variables. Certain variables like `gndr` (gender) and `hhlio12` (live with child under 12) were straightforward binary categorical variables. These were cleaned of special codes and encoded manually:

```
df_train['gndr'] = df_train['gndr'].replace({9: np.nan})
# Encode as binary: 0 = Male, 1 = Female
df_train['gndr'] = df_train['gndr'].map({1: 0, 2: 1})

df_train['hhlio12'] = df_train['hhlio12'].replace({6: np.nan, 7: np.nan, 8: np.nan, 9: np.nan})
# Encode as binary: 1 = Yes, 0 = No
df_train['hhlio12'] = df_train['hhlio12'].map({1: 1, 2: 0})
```

One-Hot Encoding for Nominal Categorical Variables were used as variables such as `marsts`, `domicil`, `health`, and most attitudinal/value scales, we used one-hot encoding to preserve interpretability and avoid false ordinality.

In simple words:

Encoding was tailored to the structure and semantics of each variable in the dataset. Rather than applying a blanket transformation, we used a hybrid strategy that supports clarity, econometric compatibility, and meaningful model interpretation. One-hot encoding produces binary columns that clearly map to original response categories (e.g., `health_2 = 'Fair'`), avoiding ambiguity and enhancing the transparency of coefficients in Logit/Probit models. This is critical when interpreting marginal effects and policy relevance.

Econometric modeling in R (via `glm()` for Logit/Probit) requires numerical matrices. All variables—whether originally binary, ordinal, or nominal—must be numerically encoded to support likelihood maximization and matrix-based operations like variance inflation factor (VIF) checks or AIC comparisons. Variables like `hinctnta` (income decile) or `stflife` (life satisfaction) possess clear ordinal properties. Encoding them as numerics preserves the natural rank order. Conversely, `domicil` (urban/rural classification) or `marsts` (marital status) have no ordinal logic and are encoded using dummies to preserve category uniqueness.

Improper encoding (e.g., treating nominal variables as ordinal integers) can introduce false assumptions into the model. Our careful distinction between ordinal and nominal variables helps mitigate this issue. Some attitudinal indicators like `ipcrtiv` (importance of being creative) were ordinal in theory but functioned more like likert-scale statements; one-hot encoding allowed modeling their influence without assuming linearity.

The final dataset structure enables direct input into Logit/Probit regression with interpretable, statistically sound, and semantically meaningful predictors. This precision in encoding ensures that downstream inferences on employment determinants are both valid and robust.

7. Final Check and Saving Clean Data

After completing all transformations, we performed a final check for missing values and structural consistency in the dataset. This included inspecting `.info()` and visualizing sample records using `.head()` to ensure the encoded structure was correctly applied.

```
FIRST few rows:
   pdwrk  agea  gndr  agepnt  ttminpnt  acchome  chldo12  hhllo12  hincsrca  \
2       0   58    0   90.0    240.0        1     1.0     0.0     1.0
5       1   46    1   65.0     30.0        1     1.0     1.0     1.0
7       1   54    0   79.0     20.0        1     2.0     1.0     1.0
22      1   50    1   75.0     20.0        1     2.0     1.0     1.0
23      0   48    0   69.0     15.0        1     2.0     0.0     1.0

   marsts_2.0  ...  stflife_1.0  stflife_2.0  stflife_3.0  stflife_4.0  \
2      False  ...      False      False      False      False
5      False  ...      False      False      False      False
7      False  ...      False      False      False      False
22     False  ...      False      False      False      False
23     False  ...      False      False      False      False

   stflife_5.0  stflife_6.0  stflife_7.0  stflife_8.0  stflife_9.0  \
2      False      False      False      True      False
5      False      True      False      False      False
7      False      False      True      False      False
22     False      False      False      True      False
23     False      False      False      True      False

   stflife_10.0
2      False
5      False
7      False
22     False
23     False

[5 rows x 76 columns]
```

The printed structure confirmed that all transformations had been applied correctly:

- Binary variables such as `gndr` and `hhllo12` were properly mapped (0/1).
- Multi-level categorical variables like `marsts`, `stflife`, and `hincsrca` were one-hot encoded, generating distinct binary columns per level.
- The resulting DataFrame included a total of 76 columns — up from the original 23 — due to the expansion introduced by dummy variables.

- The following visual inspection confirmed that each encoded variable was transformed into Boolean columns with True/False values, where True indicates membership in that specific category.
- stflife_8, stflife_9, and stflife_10 columns represent different levels of life satisfaction, marsts_2.0 corresponds to a marital status alternative to the reference category, All False/True columns are mutually exclusive within each group, consistent with dummy coding logic.

This structured and interpretable design ensures clarity in regression interpretation, especially when comparing outcomes across multiple categories. Once satisfied with encoding and structure, we performed the final missing data cleanup.

The final DataFrame comprises 18,646 observations and 76 columns, serving as the foundation for our diagnostic and estimation pipeline using Logit and Probit models. Although imputation is a common practice, we elected to perform complete-case analysis to ensure consistency across both frameworks. Additionally, the finalized DataFrame has been saved to a new CSV file, facilitating reproducibility and future analyses.